

NAK — GitHub as an Underused Non-Linear Almanac Zine Host

Repositories as living public thought, issue-books, repair ledgers, and weird little rooms

2026-07-06

★ ETCHA

♡ CHOOCH

✦ SNAK

YIP!

red-X

sullybox

watercooler

bug report form

FLASHI trail

NAK — GitHub as an Underused Non-Linear Almanac Zine Host

Repositories as living public thought, issue-books, repair ledgers, and weird little rooms

front-door sticker

The repo is not just where the artifact lives.

The repo is the artifact's weather history.

Abstract

This paper argues that GitHub is an underused host for non-linear almanac zines: living public knowledge objects where version history, issues, branches, releases, forks, and repair trails are not backstage infrastructure, but part of the publication form itself. GitHub is usually read as a software forge. That reading is true but too small. A repository can also operate as a public workshop, annotated archive, zine press, almanac engine, field notebook, errata room, and non-linear book. The underuse is therefore a genre failure, not a platform failure. The paper names the repo-zine grammar, gives a design template, and warns against failure modes: dead repos, maze repos, authority theatre, platform dependence, accidental leakage, and source-display cosplay.

repo = zine body

README = cover

folders = rooms

commits = dated marginalia

issues = letters

releases = numbered issues

1. Object and H0

Object: GitHub as a host for living, non-linear, dated, public knowledge forms.

H0: GitHub is only being used as intended: code storage, version control, software collaboration, and static site hosting. The zine/almanac reading is decorative metaphor.

Working claim: H0 fails if the platform's ordinary mechanics already perform the work of zines and almanacs: self-publication, partial circulation, dated recurrence, reader participation, marginal repair, issue-by-issue release, visible route-record, alternate editions, and open correspondence.

The claim is not that GitHub was secretly built for zines. It is that the machinery that makes code accountable also makes public thought repairable.

OBJECT GATE: artifact ≠ final PDF only.
The workshop can be part of the publication.

2. GitHub beyond code

GitHub's public identity is code: repositories, commits, branches, issues, pull requests, releases, and collaboration tools. The platform's own documentation still gives us the wider grammar. A commit records changes to files and attaches a unique SHA, time, author, and message; this is already dated marginalia, not just engineering metadata.¹ Branches support contained experiments; this is already alternate-edition logic.² Forks are independent copies that can propose changes upstream; this is already a sibling-zine and submission route.³

A repo therefore has two bodies:

CODE BODY:

files, tests, build, deploy

PUBLICATION BODY:

cover, rooms, marginalia, letters, errata, editions, forks, seasonal issues

Most people only name the first. The second is sitting in the same interface.

3. Zines as living, partial, modular public objects

Zines are not merely small magazines. They are self-published, low-barrier, partial, circulating knowledge forms. In archival scholarship, zines have been treated not only as artifacts to collect but as community-archive practice: participation, shared stewardship, multiplicity, activism, reflexivity, affect, and collaborative reminiscence can be part of the form.⁴ Work on zines and visualization also treats zines as a free-

form medium with room for intimate expression, marginal voices, and graphical experiment.⁵

That matters because a repo-zine should not imitate a glossy journal. It should preserve zine properties:

```
low-cost publishing
visible hand
partial issue
local vocabulary
open repair
reader reply path
scrapbook structure
dated recurrence
```

A PDF can imitate zine aesthetics. A repo can host zine behaviour.

bedroom shelf note

In the local archive, “staff”, “watercooler”, “robot_bugs_and_frogs”, “.pls”, “sullybox”, “red-X”, and “bug report form” already behave like rooms, posters, playlists, repair notes, and warning stickers. Those names are not decoration after the fact; they are interface hints.

4. Almanacs as dated, recurring, seasonal knowledge forms

An almanac is an annual or regularly recurring reference form: calendars, astronomical or meteorological data, tables, statistics, and miscellany for a given year or subject.⁶ The almanac is useful here because it is not only a book. It is a time-indexed knowledge habit.

A repo can do almanac work better than a flat web page:

```
/almanac/2026/july.md
/almanac/2026/autumn.md
/releases/v2026.07
/issues?q=label:seasonal
/tags/solstice-issue
/commits/2026-07-06-weather-turn
```

GitHub’s releases can be attached to tags, named, described, drafted, published, and supplied with release notes; a release can therefore function as a numbered zine issue or seasonal almanac edition.⁷ GitHub’s permanent links let a reader pin a file to a specific commit rather than a moving branch head; that gives almanacs a frozen sky-map when needed.⁸

5. The repo-zine grammar

The repo-zine grammar is not invented from nowhere. It is ordinary GitHub machinery re-read as publication machinery.

Repo part	Zine / almanac role	Why it matters
README.md	cover, contents, front matter	Gives the first threshold and voice.
folders	rooms, sections, cabinets	Supports non-linear reading without pretending there is one correct order.
commits	dated marginalia, weather log	Shows when and how the object changed.
issues	letters, errata, open questions, public notebook	Lets uncertainty sit in public without pretending to be finished.
labels	coloured tabs, shelf marks	Turns the issue list into a finding aid.
pull requests	submissions, contested edits, public repair	Makes revision visible and attributable.
branches	alternate editions, experimental rooms	Lets an idea live before it becomes the main edition.
tags	named waypoints	Marks versions that should be cited or revisited.
releases	numbered issues, seasonal drops	Gives serial rhythm to a living object.
wiki	long-form house manual	Holds extended explanation near the repo.
GitHub Pages	reader-facing edition	Publishes a browsable surface without separating it from source.
forks	sibling zines, local editions	Lets readers copy, annotate, diverge, and return.

GitHub’s issue and pull-request filters support labels, assignees, types, fields, review status, and nested queries; those filters can be used as a reader’s map, not just a maintainer’s task sorter.⁹ Wikis explicitly support long-form content about how a project is used, designed, or principled.¹⁰ GitHub Pages publishes static files from a repository, optionally through a build process; the important point is not “make a website,” but “make a reader-facing edition whose backstage remains inspectable.”¹¹

ISSUE = letter

PR = contested paste-up

TAG = bookmark

FORK = sibling issue

6. Why PDFs and blogs flatten this shape

PDFs are excellent for stable handoff. Blogs are excellent for linear public reading. Neither is ideal for a living non-linear object.

They flatten in different ways:

```
PDF:
  good for citation, bad for visible repair

blog:
  good for reading, weaker for route-record and branching

folder of notes:
  good for writing, weak for public issue/repair trail

repo-zine:
  preserves artifact + repair + history + alternate routes
```

The problem with many public methods is not that they lack a final document. It is that the route to the document disappears. GitHub can keep the route alive: the wrong turn, the erratum, the issue that became a section, the branch that did not merge, the release that got superseded, the fork that made a better local edition.

This is what “weather history” means. The repo does not only show the artifact’s present surface. It shows weather, damage, repair, and season.

7. Example template: a repo-zine starter house

```
repo-zine/
  README.md                # cover, doorway, current issue
  START_HERE.md            # reader route
  LICENSE.md               # sharing rule
  CHANGELOG.md             # weather history digest
  CITATION.cff             # citeable identity
  /almanac/
    2026-07.md             # dated issue
    2026-08.md
  /rooms/
    kitchen_table.md       # informal notes
    bug_wall.md            # known errors
    field_notes/
      diagrams/
  /letters/
    call_for_submissions.md
    reader_replies.md
  /evidence/
    source_map.md
    source_rent_register.md
  /errata/
    open_errata.md
    repaired_errata.md
  /stickers/
    ETCHA.md
    CHOOCH.md
    SNAK.md
  /.github/
    ISSUE_TEMPLATE/
      field-note.yml
      erratum.yml
      reader-letter.yml
      source-rent.yml
    pull_request_template.md
  /docs/
    index.md               # GitHub Pages surface
```

Suggested labels:

```
erratum
reader-letter
field-note
source-rent
seasonal
room-map
needs-witness
alternate-edition
repair-done
issue-zero
```

Suggested release pattern:

```
v2026.07  — July room issue  
v2026.08  — August room issue  
v2026.autumn — seasonal almanac drop
```

**MOGRI FOR REPO-ZINES: hold the object, not only the platform.
A repo-zine is a method of custody, not a GitHub aesthetic.**

8. Failure modes

A repo-zine can fail.

8.1 Dead repo

A repo that advertises living process but never updates becomes a staged bedroom. Repair: mark status honestly. Archive, dormant, active, seasonal, or finished.

8.2 Maze repo

Too many folders without a reader route makes the repo hostile. Repair: `START_HERE.md`, a room map, and labels that behave like tabs.

8.3 Authority theatre

Source density can impersonate thought. A repo full of links and release notes can still fail object custody. Repair: every source needs a role: evidence, counterexample, route marker, history, or decoration.

8.4 Archive rot

Links die, branch heads move, assets vanish, and dependencies decay. Repair: tags, releases, permanent links, local source notes, and periodic shelf audits.

8.5 Private leakage

Repos tempt publishing by copy-paste. That can leak names, prompt bodies, private correspondence, or unsafe detail. Repair: public/private boundary file, redaction pass, and delayed release branches.

8.6 Platform dependence

A repo-zine is not the same as GitHub worship. GitHub is useful because it makes certain structures cheap. The genre should survive export to GitLab, Codeberg, local Git, static files, or printed issue bundles.

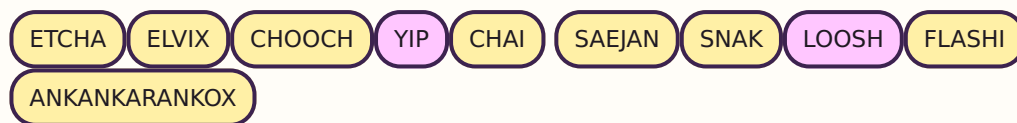
9. Local shelf field note: the repo already behaves like a roomed zine

This paper's decoration comes from the local Yaiyip.zip archive as a design stimulus, not as external evidence. The scan found 12 Git repositories, 1,071 non-Git files, and a strange, useful mixture of .txt, .md, no-extension files, .docx, images, .zip, .pls, .etcha, .chooch, and other small local formats. Top-level shelves included lmxdi, mogri, robot_bugs_and_frogs, CSP-106, staff, amphi, dragi, lexii, storyforge101, system_witch, lumixdeee, and transwhatification.

That is already a repo-zine lesson:

```
staff/README.md      = foyer note
staff/tree.md        = generated link map
staff/watercooler/    = social room
staff/sullybox/       = warning box
robot_bugs_and_frogs/ = bug shelf + conlang cabinet
Elven_Base_Alpha/Elvish/ = sticker drawer
Current_Robot_Bugs/   = issue wall
Fixed_Robot_Bugs/     = repaired issue wall
```

The local archive also shows a major design risk: charm can outrun wayfinding. A bedroom can be gorgeous and still impossible to find your socks in. Repo-zine design needs delight and route discipline at the same time.



10. Design rules for a repo-zine

Rule 1: publish the workshop, not only the outcome

The finished essay matters. The repair ledger also matters. The issue that changed the question may matter more than the final paragraph.

Rule 2: give every room a door sign

Non-linear does not mean unmapped. Every folder should say what it is, who it is for, and what not to expect inside.

Rule 3: treat releases as issues, not only downloads

A release can be a numbered zine issue with notes, assets, and a known date. The release note should tell readers what changed, what remains disputed, and what to read first.

Rule 4: keep open questions visible

Do not hide all uncertainty in private notes. Use issues for questions that readers may help answer, disagree with, or cite as unresolved.

Rule 5: separate source rent from decoration

A sticker can be a sticker. A source must pay rent. Put decorative material in /stickers or /wall, and evidential material in /evidence.

Rule 6: cite the frozen sky when needed

Branch URLs move. Commit permalinks and tags give readers the version you actually saw. Use them for claims that need exactness.

Rule 7: let forks be local editions

A fork is not only a technical copy. It can be a local edition: annotated, translated, rearranged, adapted, then optionally sent back as a pull request.

11. What this gives academic and public writing

Repo-zines are especially useful for:

```
public methods
living bibliographies
field notebooks
community archives
open peer commentary
research diaries
repair logs
issue-based teaching
conlang/documentation projects
local history almanacs
weird knowledge gardens
```

Academic publishing often hides route, uncertainty, failed branches, reviewer scars, and version weather. A repo-zine can keep those visible without forcing every reader to walk through them. The reader can take the front edition, the issue wall, the source map, or the full weather history.

This is not anti-PDF. It is anti-PDF-only.

12. Conclusion

GitHub is underused because the genre has been misread. People see a software repository. They miss the public workshop, annotated archive, zine press, almanac engine, repair ledger, and non-linear book.

The repo-zine is not a gimmick. It is a publication form suited to objects that change, argue, recur, fork, and repair themselves in public. It lets the artifact keep its weather.

Publish the workshop, not only the finished object.
Let the repo be the zine body, the issue wall, the almanac table, and
the repair ledger.

References

Appendix A: pocket scorecard for repo-zines

Gate	Pass question
Object	Does the repo say what object it holds?
Route	Can a new reader find the front door, rooms, and issue wall?
Time	Are versions, tags, releases, or dated notes meaningful?
Repair	Are errors and changes visible?
Participation	Can a reader reply, submit, or fork without guessing the rules?
Source rent	Are evidence and decoration separated?
Export	Can the thing survive outside GitHub if needed?
Safety	Is the public/private boundary named?

Appendix B: sticker sheet

ETCHA = language/agency sticker
ELVIX = audit/control sticker
CHOOCH = machine-facing runtime sticker
YIP = yes/fork/thank-you sticker
CHAI = carry-over/change/chat sticker
SAEJAN = perhaps sticker
SNAK = bridge network sticker
LOOSH = water-to-horizon sticker
FLASHI = trail sticker
ANKANKARANKOX = reassess / do not eat the object sticker

1. GitHub Docs. "About commits." <https://docs.github.com/en/pull-requests/committing-changes-to-your-project/creating-and-editing-commits/about-commits>↵
2. GitHub Docs. "About branches." <https://docs.github.com/articles/about-branches>↵

3. GitHub Docs. "About forks." <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/about-forks>↵
4. Baker, S. "Zines as community archive." *Archival Science* 22, 539–561 (2022). <https://link.springer.com/article/10.1007/s10502-022-09388-1>↵
5. McNutt, A. "On the Potential of Zines as a Medium for Visualization." arXiv (2021). <https://arxiv.org/abs/2108.02177>↵
6. Merriam-Webster. "Almanac." <https://www.merriam-webster.com/dictionary/almanac>↵
7. GitHub Docs. "Managing releases in a repository." <https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository>↵
8. GitHub Docs. "Getting permanent links to files." <https://docs.github.com/en/repositories/working-with-files/using-files/getting-permanent-links-to-files>↵
9. GitHub Docs. "Filtering and searching issues and pull requests." <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/filtering-and-searching-issues-and-pull-requests>↵
10. GitHub Docs. "About wikis." <https://docs.github.com/communities/documenting-your-project-with-wikis/about-wikis>↵
11. GitHub Docs. "What is GitHub Pages?" <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>↵